

ADR-009: Preview clips, cut without a decoder

Status: Accepted, implemented and **playing in the client** as of 2026-11-04. `internal/preview` cuts Ogg and Opus, `GET /song/{chart_hash}/preview.ogg` serves it, deployed on vault2 as `3a9bf3c`, and `RemoteSongEntry.LoadPreviewAudio` fetches it — verified by VLC and then by **BASS, the game's own decoder** (both below). **MP3 is cut too as of 2026-11-04** — see "*What the MP3 cutter actually fixed*" below, because the reason first written here was wrong: those 150 songs were never silent. Clips are still not cached on disk.

Context: Jay, playing the fork with the server library browsing: "*the song preview does not load*". Correct, and by design — `RemoteSongEntry.LoadPreviewAudio` returns `null`, because a song the server published and this disk does not hold has no bytes to preview. Jay asked for a server endpoint instead of a client-side download.

What the server already has

Nothing needs adding to the catalogue. Every song already carries:

Field	Source
`preview_start_ms`, `preview_end_ms`	`internal/scan/scan.go:229`, from `song.ini`
`assets.stems`	`internal/scan/scan.go:422`
stem classification	`internal/scan/names.go:87` `ClassifyStem`

And `internal/sng` implements `fs.FS`, so a stem inside a package opens as an ordinary file.

The decision

Cut the clip at container boundaries. Do not decode, do not re-encode, add no dependency.

Why not ffmpeg

This was the obvious first answer and it is wrong here, for a reason that is specific to this deployment rather than general. The runtime image is `gcr.io/distroless/static-debian12:nonroot` (`Dockerfile:19`) — **static distroless: no shell, no package manager, nothing to install into.** Adding ffmpeg means replacing the base image and carrying roughly 60–100 MB of it, against ADR-001's "tiny container" and the Raspberry Pi target that is a project promise rather

than a bonus. ADR-003 already set the precedent for pricing a dependency before taking it: `sevenzip` cost 3 → 71 packages and +1.62 MB, and that was judged worth it. An ffmpeg layer is two orders of magnitude more and would also need an arm64 build for the Pi.

Why not range-serve the stem

The cheap alternative — serve bytes *N*..*M* of the stem and let the client's BASS decoder resync — was offered and then **ruled out by measurement**, not by taste. Counted across the live catalogue, 22,091 stem files:

```
.ogg     20288
.opus     1354
.mp3       449
```

98% is Ogg-family. An Ogg file declares its codec, sample rate and channel count in header pages at the *start* of the file; hand a decoder bytes from the middle and there is nothing to decode with. Range-serving would have worked for the 449 MP3 stems and failed for the other 21,642. It is not a cheaper option, it is a broken one.

What "cut without decoding" means

An `.ogg` file is not one blob. It is a sequence of self-describing **pages**, each 27 bytes of header plus a segment table plus payload, carrying a `granule_position` that says how far into the stream the page ends. Playing audio means turning pages into sound, which needs a codec. *Cutting* audio means copying whole pages, which needs only the ability to find where pages begin and end — reading headers, not decoding them.

The same shape holds for the other two formats, which is why one approach covers all of them:

Format	Unit	How its position is known
Ogg Vorbis	page	`granule_position`, in samples at the rate in the identification header
Ogg Opus	page	`granule_position`, always in 48 kHz units regardless of input rate
MP3	frame	duration from the header's bitrate and sample rate, found by the sync word
WAV	byte	raw samples; pure arithmetic — **not implemented, see below**

So the algorithm, for the 98% case:

1. copy the leading pages whose `granule_position` is 0 — those are the headers (`\x01vorbis` identification plus comment and setup, or `OpusHead` plus `OpusTags`), and copying them is what makes the output a standalone playable file rather than a fragment
2. read the sample rate out of the identification packet, so granule positions convert to time
3. skip pages until `granule_position >= preview_start`
4. copy pages until the clip is long enough
5. set the EOS flag on the last page, renumber the page sequence, and **recompute the CRC** of any page whose header was touched

Output: a valid Ogg of a few hundred KB instead of a ~50 MB download.

What this costs, stated honestly

It is a parser, and parsers are where this project has found most of its defects. Two known wrinkles rather than surprises:

- **Vorbis uses overlapping windows**, so the first audio page of a cut clip has no predecessor to overlap with and the first ~20 ms can glitch. Acceptable for a preview; worth knowing before somebody reports it as corruption.
- **Granule positions must be rebased** so the clip starts near zero, or a player reads it as a seek into a longer stream and may report a wrong duration.

How it must be verified, and what would not count

A clip this code calls valid is not a clip somebody can play. The repo already has this rule in two forms: *"a forgiving reader hides a strict writer, so exercise a new container by EXTRACTING it"*, and *"round-tripping against our own reader only proves our reader and writer agree with each other."* Both apply exactly. A cutter and a parser written in the same afternoon will agree with each other about a malformed page.

So the standard is an **independent decoder**: the clip must play in the real YARG client, which is the oracle this project already uses for packaging questions, and `BASS` is not our code. A Go test asserting page structure is necessary and is not sufficient, and must not be reported as proof the feature works.

The unit tests build Ogg streams by hand — pages with known granule positions — because that tests the page arithmetic precisely and needs no fixture. Per the repo's rule, the cut must be seen to go **wrong** first: a test that cuts at a known boundary and asserts the wrong page count should fail before the correct one is trusted.

The endpoint

```
GET /song/{chart_hash}/preview.ogg
```

- 404 when the hash is unknown, as `/song/{hash}.sng` already does
- 415 when the stem is a format this cannot cut
- strong `ETag`, because the output is deterministic — the same package and the same `preview_start_ms` always yield the same bytes, which is the property ADR-002 and the packer's deterministic mask already protect
- cached on disk beside the packed archives, so a clip is cut once

Which stem. `ClassifyStem` can report several. The preview should use the full mix where a song has one, and otherwise the loudest-weighted guitar or vocals stem rather than an arbitrary first entry — mixing stems would mean decoding, which is the one thing this design avoids. That choice is a correctness question for the oracle, not a guess to be made here.

Not in scope

- **Mixing multiple stems.** Needs a decoder. A single stem is a worse preview than YARG's own and is still a preview.
- **Transcoding to one format.** Clients get the container the stem was in. YARG decodes all three already.

- **Generating previews at ingest.** On-demand plus a cache gets the same result without a migration or a rescan of 4,085 songs.

Verified, and by what

A clip cut from the live library's own packages, served by the real endpoint, then decoded by **VLC's libvorbis** — not this project's code:

```
GET /song/a56c61f1…/preview.ogg   ->  200, 557,663 bytes in 10 ms   (the stem is 47 MB)
ETag: "7cde27f2…-preview-30000-30000"
Content-Type: audio/ogg

independent Ogg walk:  130 pages, 0 CRC mismatches, BOS on first, EOS on last,
                       granules rebased 0..1331144, sequence contiguous, 2 header pages
VLC decode:            30.16 s, 44.1 kHz stereo 16-bit
audio content:         peak 0.0 dBFS, rms -12.4 dBFS, audible in the first AND last second
```

Four things in that list are there because each would have passed without the others:

- **the CRC was checked by a second implementation**, in Python, written against the spec rather than by reusing the Go code under test
- **the header packets were checked for spill.** If Vorbis's setup packet had run past the granule-zero pages, the clip would have lost its codebooks and been undecodable while passing every structural assertion. Page 1's last segment is 145, not 255, so the packet ends there and page 2 starts fresh.
- **it was decoded, not just parsed.** A cutter and a parser written in the same afternoon agree with each other about a malformed page.
- **the decoded audio was measured for silence.** Thirty seconds of silence decodes perfectly and is not a preview.

The clip is byte-identical across a rebuild and a handler rewrite, which is the determinism the strong ETag depends on.

And it has now met BASS — 2026-11-04. This paragraph used to say the oracle had not had its say, and that VLC was strong evidence rather than the final word. It has had its say. `RemotePreviewProbe` calls `RemoteSongEntry.LoadPreviewAudio` — the production path, the one the music library calls when a song is highlighted — against the live server, and hands what comes back to the game's own decoder:

```
PROBE INFO: Fredrik Thordendal's Special Defects - "Solo" Niger Within
PROBE PASS: the song has no bytes on this disk, so this is the remote path
PROBE INFO: BASS decoded it in 619 ms, length 30.06s, 1 channel set(s)
PROBE PASS: the clip has real length, not zero
PROBE PASS: and it is about the preview window rather than the whole song
PROBE PASS: a song the server does not have yields no preview
```

619 ms covers the fetch **and** the decode, against a 47 MB stem that was never downloaded.

The length assertions are not padding: a decoder handed rubbish can still return an empty stream, and that would satisfy "returned a mixer" perfectly well. The last line is the negative control — without it, a loader that returned some cached or default audio for any hash would pass everything above it.

And Jay heard one, in the game, on a song this machine does not hold — `playtest-2026-11-04`, the same evening. That is a different claim from the probe's and it is the one that finishes the feature: a probe can only say BASS accepted the bytes, not that a person highlighting a song in the music library hears the right music at the right moment. Every link now has evidence of its own — the cutter against real encoders' CRCs, the clip against VLC, the client against BASS, and the whole of it against somebody's ears.

What the implementation does that this document originally did not say

- **Candidates are tried in order, not picked once.** A test caught a song whose full mix could not be cut returning 500 while a usable guitar stem sat beside it in the same package. Each skip is logged with its reason, and when every candidate fails the request still fails.
- **The ETag is over the PACKAGE hash**, not the chosen stem's, because which candidate wins is decided after the tag has to be sent — and any stem changing changes the package hash anyway.
- **The disk cache is not built.** Clips are cut per request that the ETag does not satisfy. A performance gap, not a correctness one, and recorded here rather than left to be discovered from a slow endpoint.

MP3, added 2026-11-04

What the MP3 cutter actually fixed, and what this entry claimed first

The justification originally written here was false, and it was written without being checked against the running server. It said 150 of the 4,085 songs carry only `.mp3` stems, answered 415, and played nothing. The first half is right — 150 songs, measured from the catalogue's own `assets.stems`, and 0 songs carry a `.wav` stem, which is why no WAV cutter exists. The rest was an inference, and production contradicts it:

```
200  ct=audio/ogg  len=482304  magic=ff fb 94 64   Motorhead - (We Are) The Road Crew
200  ct=audio/ogg  len=398735  magic=ff fb 90 64   Queens of the Stone Age - 3's & 7's
200  ct=audio/ogg  len=338549  magic=ff fb 90 64   Foo Fighters - All My Life
```

`ff fb` is an MP3 frame header. All 150 ship a ready-made `preview.mp3`, `previewCandidates` puts a ready-made preview **first**, and `ReadyMade` deliberately skips the extension filter — so the 415 never happened. **215 of 4,085 songs carry their own preview file** (150 mp3, 63 ogg, 2 opus), and the comment in `previewCandidates` asserting "*Nothing in the live library does - 0 of 4,085*" was stale; it was read as a current measurement.

So on this library the change delivers exactly one thing: those 150 responses stop being labelled `audio/ogg` when they are mpeg. `CutMP3` **runs on zero songs here**, because no song has an MP3 stem *and* no ready-made preview. It is correct, verified, and insurance for a library that has one — which is a smaller claim than the one it shipped under.

The error was not in the measurement that was taken; it was in not taking the one that mattered. A stem extension says what the archive holds, not what the endpoint answers, and the endpoint is one HTTP request away.

How it works

Same idea, different container: an MP3 is a row of self-contained frames, so a clip is a run of whole frames copied out. Frame length comes from the header's bitrate and sample rate, which is also why it has to be parsed per frame rather than assumed — VBR files change bitrate as they go.

Two things that are easy to get wrong and silent when you do:

- **MPEG2 and 2.5 use different constants.** 576 samples per frame rather than 1152, and `72` rather than `144` in the length formula. Using the MPEG1 numbers on an MPEG2 file yields frames of twice the right length and a clip of noise.
- **An ID3v2 tag's size field is syncsafe** — seven significant bits per byte. Read it as plain big-endian and you overshoot into the audio. The resync would recover, but by discarding real audio, so the clip would start late for no visible reason.

Verified against a real encoder's file, and the first version of the check did not discriminate. The test walks the `.mp3` stem of a song pulled from the live server and asserts the frames *tile* it:

```
guitar.mp3: 2,641,504 bytes, 6,320 frames, 100.0% covered, 0 bytes skipped
```

The control that matters: changing `144` to `143` in the length formula still covered **99.3%** and **passed** the original 95% threshold, because resync recovers within a few bytes of each frame. The assertion is now on *skipped bytes*, where correct is zero and the corrupted formula skips 12,639:

```
corrupted: 99.5% covered, 12639 bytes skipped  -> FAIL
restored : 100.0% covered,     0 bytes skipped -> pass
```

That is the whole lesson of the entry: **coverage was a plausible metric that could not see the defect**, and only putting the bug back revealed it.
